A Registered 4-bit Array Multiplier from RTL to GDSII

Vedant Zope (Roll No. 2024102003) Kavya Pandey (Roll No. 2024102001)

Digital VLSI Design Project

Abstract

A registered unsigned 4×4 array multiplier was implemented in Verilog and taken through synthesis, timing analysis, and physical design using the SKY130 high-density standard-cell library. The RTL contains 16 partial-product AND operations, 12 full-adder instances, and 16 flip-flops. Exhaustive simulation checks all 256 operand pairs. Synthesis produces 70 cells occupying 797.0144 µm^2; flip-flops account for 50.24% of this area. The saved typical-corner timing report places the minimum-period boundary near 2.983 ns, corresponding to approximately 335.23 MHz. The final 150 µm square layout uses a 6.5 ns clock target and has positive setup slack, zero reported DRC errors, and zero LVS errors. Its worst-corner frequency estimate is 166.65 MHz. The report distinguishes measured artifacts, derived values, and missing evidence, including remaining slow-corner slew violations.

1 Scope and evidence

The aim is to verify the multiplier's arithmetic and registered behavior, map it to the SkyWater 130 nm `sky130_fd_sc_hd` library, and explain the area and timing of its final layout. The assignment's Parts A–D and six report questions define the required content. This report uses the requested two-column format. The assignment does not specify a font, page limit, or a different report section template.

The inspected material consists of the supplied assignment PDF, the project folders `rtl`, `tb`, `synth`, `sta`, and `pnr`, and 13 pictures. The two original ZIP containers are unavailable in the workspace; the extracted files are the available source material. The RTL, mapped netlist, statistics, STA report, configuration, metrics JSON, and GDS are primary evidence. Terminal, waveform, and layout screenshots provide visual evidence. Six pictures containing explanatory prose or prefilled tables are reference notes, not independent tool measurements. Their numerical claims were checked against the primary files. Copies and SHA-256 hashes are provided in the accompanying source package.

The original RTL and testbench were replayed with Icarus Verilog. The required fault-injection test was also run on a temporary copy. Synthesis, standalone STA, and physical design were analyzed from the supplied outputs; the original PDK and those executables are not available in this reporting environment, so those stages are not claimed as new runs. The netlist header identifies Yosys 0.62; the other original tool and PDK versions are not recorded. The supplied terminal images show a Nix-shell prompt; the assignment's suggested Docker setup is not assumed to be the recorded execution environment.

2 Part A: RTL design and verification

2.1 Implementation and register boundaries

The top module, `mult_array`, has scalar inputs `clk` and `rst_n`, 4-bit inputs `a` and `b`, and the registered 8-bit output `p`. On every rising edge, `a_reg` and `b_reg` capture the operands while `p` captures the product formed from the preceding registered operands. Active-low asynchronous reset clears all three register banks. Thus the design has $4+4+8 = 16$ flip-flops and no arithmetic pipeline registers inside the array.

Each partial product is

$$pp_i[j] = a_{\mathrm{reg}}[j] \wedge b_{\mathrm{reg}}[i], \qquad i, j \in \{0, 1, 2, 3\}. \quad (1)$$

There are 16 such AND operations. Row 0 supplies product bit 0 directly. Three rows of four `full_adder` instances add the remaining products. The full-adder module uses continuous assignments:

$$s = a \oplus b \oplus c_{\mathrm{in}}, \quad (2)$$

$$c_{\mathrm{out}} = (a \wedge b) \vee [c_{\mathrm{in}} \wedge (a \oplus b)]. \quad (3)$$

The multiplication operator does not appear in the synthesizable RTL; it is used only by the testbench to calculate the expected answer.

Table 1: Actual array interconnections. In every adder row, $c_i[0] = 0$.

Row	Connections and completed product bits
0	$r_0 = \{0, pp_0[3:1]\}$; $p_{\mathrm{comb}}[0] = pp_0[0]$
1	$s_1, c_1 = pp_1 + r_0$; $p_{\mathrm{comb}}[1] = s_1[0]$; $r_1 = \{c_1[4], s_1[3:1]\}$
2	$s_2, c_2 = pp_2 + r_1$; $p_{\mathrm{comb}}[2] = s_2[0]$; $r_2 = \{c_2[4], s_2[3:1]\}$
3	$s_3, c_3 = pp_3 + r_2$; $p_{\mathrm{comb}}[3] = s_3[0]$; $p_{\mathrm{comb}}[7:4] = \{c_3[4], s_3[3:1]\}$

2.2 Exhaustive checks and error detection

The supplied testbench holds reset low across the first two rising edges, then applies nested loops over $a, b = 0, \ldots, 15$. For each pair it waits for two rising edges and then delays one more simulation unit before comparing with $a \times b$. This avoids sampling before the nonblocking register assignments have updated. A case-inequality comparison also treats unknown output bits as errors.

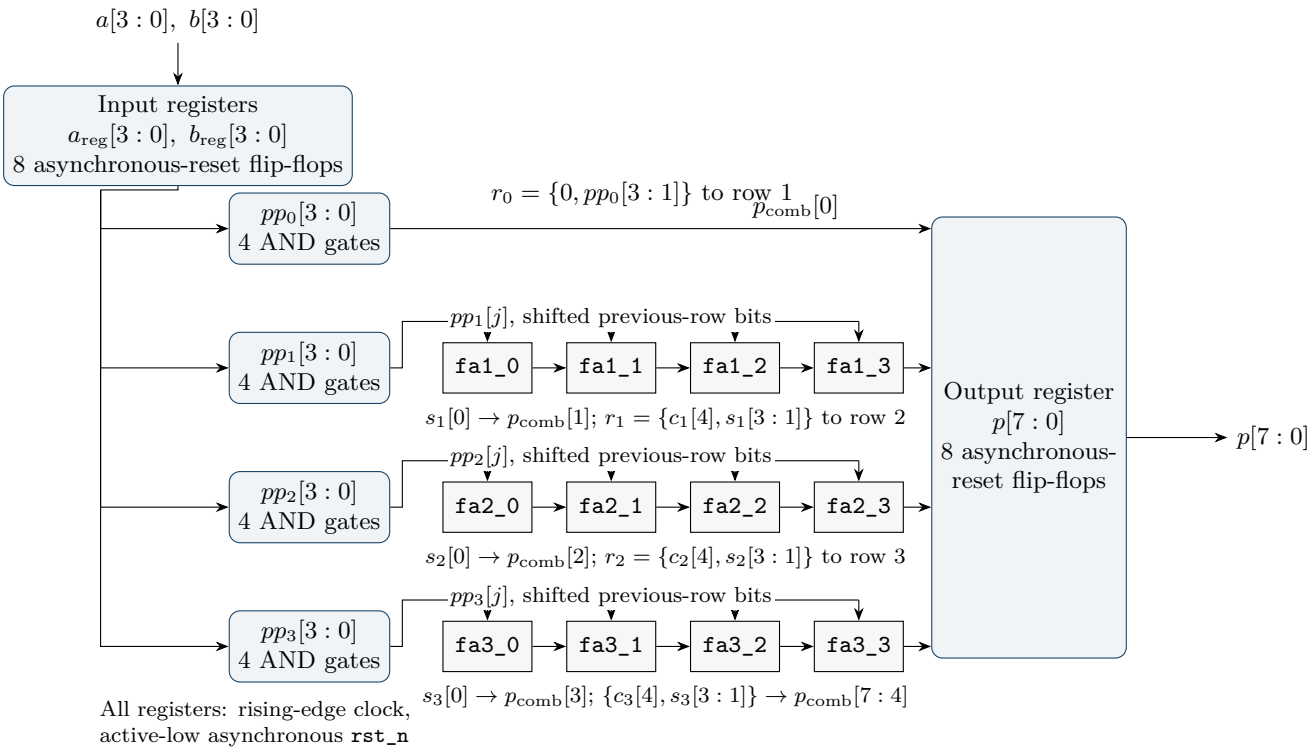


Figure 1: Block diagram reconstructed from the delivered RTL. Each AND bank contains four gates, and the twelve full-adder boxes correspond to the actual instances. Carries ripple left to right within each row. The shifted row bus passes the upper three sums and final carry to the next row; the exact bus equations are given in table 1. The output bank collects all eight product bits.

Every mismatch is printed, followed by one PASS or FAIL summary; the waveforms are saved as `mult.vcd`.

The supplied RTL screenshot and the new replay both give

```
PASS: 256 vectors checked, 0 errors
```

To test the checker, only the sum expression in a temporary copy of `full_adder.v` was changed to `a ^ b`. The replay gives

```
FAIL: 256 vectors checked, 95 errors
```

For example, 3×3 should give 9 but gives 5 with this fault. This shows that the checker detects the lost carry-in contribution. The delivered RTL was preserved, and its passing run is separate from the fault experiment. The full mismatch log and replay commands accompany the report.

2.3 Worked example and waveform latency

The example uses **Kavya Pandey's roll number 2024102001**. Its last digit is 1, so $A = 1 + 5 = 6$ and $B = 7$. The alternative $B = 9$ rule is not needed because $A \neq 7$. The binary calculation follows the assignment's shift-and-add layout:

00000110	$A = 6$
$\times$ 00000111	$B = 7$
00000110	$pp_0 = 6$
00001100	$pp_1 \ll 1 = 12$
00011000	$pp_2 \ll 2 = 24$
00000000	$pp_3 \ll 3 = 0$
00101010	$P = 42$

Equivalently, $6 + 12 + 24 = 42$. Internally $r_0 = 3$, row 1 adds $6 + 3 = 9$, row 2 adds $6 + 4 = 10$, and row 3 adds $0 + 5 = 5$. Their completed bits combine to `00101010`.

There is one source-level timing issue: `always ##5 clk = ~clk` defines a 10-unit period, but none of the delivered Verilog files declares a `timescale`. The original waveform therefore shows seconds, not ns. For the new normalized replay, a separate compilation-unit file containing `` `timescale 1ns/1ps `` was compiled first. This leaves the submitted files unchanged and makes the intended clock period 10 ns. Both the original and normalized exhaustive runs pass.

The assignment calls this a two-cycle latency because it counts the two capture edges. The elapsed time *between* E1 and E2 is one period; from application just after the preceding edge to output capture it is almost two periods. The requested latency tables below retain the assignment's $2T$ convention. Once filled, the hardware can accept one operand pair per cycle. The supplied testbench verifies each pair with a two-edge

```
[nix-shell:~/librelane/dvd]$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.
v
[nix-shell:~/librelane/dvd]$ vvp sim
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:58: $finish called at 5136 (1s)
```

Figure 2: Supplied Icarus terminal output: all 256 RTL vectors pass. The `(1s)` finish-time suffix reflects the missing explicit timescale in the original source, rather than the intended nanosecond clock.

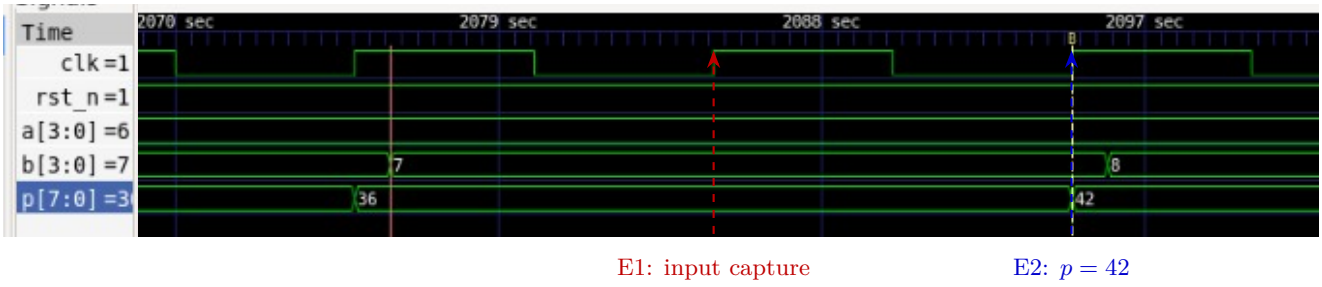


Figure 3: Supplied GTKWave evidence: aligned crops of the signal names and the example's time window retain `clk`, `rst_n`, `a`, `b`, and `p` at a readable scale. The two capture edges are marked. With $a = 6$, b changes from 6 to 7 at time 2076. E1 is the rising edge at 2085, and E2 is the rising edge at 2095, when p changes from 36 to 42. The screenshot labels these units as seconds because the original testbench has no timescale. A separate replay with an explicit nanosecond directive confirms the same event sequence at the times in table 2.

Table 2: Example events from the normalized replay's VCD.

Time (ns)	Observed event
2076	$a = 6$, $b = 7$ applied; previous product is 36
2085	E1: input registers capture 6 and 7
2095	E2: output register changes to 42
2096	Testbench samples and confirms $p = 42$

Table 3: Part B results from `mult_array_stat.rpt`; cell names below omit the common `sky130_fd_sc_hd__` prefix.

Metric	Value
Total cells	70
Flip-flops, `dfrtp_1`	16
Combinational cells	54
Total cell area (μm^2)	797.0144
Flip-flop area (μm^2)	400.384
Flip-flop area fraction	50.24%
Three most-used types	`dfrtp_1`: 16; `nand2_1`: 10; `clkinv_1`: 6
Netlist simulation	PASS: 256 vectors, 0 errors

wait, so it does not itself exercise a continuous one-pair-per-cycle input stream.

3 Part B: logic synthesis

The script `synth/synth.tcl` reads both RTL modules, performs `synth -top mult_array -flatten`, maps registers with `dfflibmap`, maps combinational logic with `abc`, and removes unused wires using `opt_clean -purge`. It then writes the statistics and mapped Verilog netlist. The Liberty file is the SKY130 HD typical corner, `tt_025C_1v80`: 25 °C, 1.8 V. The command supported by the script is `yosys -c synth/synth.tcl`.

The sequential area is $16 \times 25.024 = 400.384\,\mu m^2$. Its fraction is

$$\frac{400.384}{797.0144} \times 100 = 50.24\%. \tag{4}$$

The remaining logic occupies $396.6304\,\mu m^2$. Flattening and Boolean optimization replace the structural adder hierarchy with combinations of NAND, NOR, XOR/XNOR, majority, and complex AND–OR cells. The physical cells therefore cannot be counted as one cell per full-adder instance or one preserved cell per partial-product AND.

The supplied gate-level command combines the testbench and mapped netlist with both SKY130 model files: `primitives.v` and `sky130_fd_sc_hd.v`. It enables `-DFUNCTIONAL` and `-DUNIT_DELAY=##1`. It prints `PASS: 256 vectors checked, 0 errors`. This is exhaustive functional evidence for the mapped design at the testbench's sampling points. It is not a post-route delay simulation; timing is assessed separately by STA. The PDK models were not supplied here, so this gate-level result is attributed to the original screenshot rather than to a new replay.

4 Part C: static timing analysis

4.1 Constraints and minimum-period estimate

The supplied `sta/sta.tcl` reads the same typical Liberty corner and mapped netlist, links `mult_array`, and creates `clk`. It applies 1.0 ns input delay to all inputs except the clock and 1.0 ns output delay. It requests worst maximum slack and saves maximum-delay checks. There are no extracted interconnect parasitics or propa-

```
[nix-shell:~/librelane/dvd]$ export CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls
warning: Found both default and explicit timescale based delays. Use
       : -Wtimescale to find the design element(s) with no explicit
       : timescale.
VCD info: dumpfile mult.vcd opened for output.
PASS: 256 vectors checked, 0 errors
tb/tb_mult.v:58: $finish called at 5136000000000000 (1ps)
```

Figure 4: Supplied gate-level simulation evidence. The mapped netlist passes the same 256-vector test using SKY130 models. The displayed mixed-timescale warning limits interpretation of absolute simulation time, but does not change the reported arithmetic PASS result.

gated clock-tree delays in this standalone report. OpenSTA supports these netlist, Liberty, constraint, and timing-report operations [6].

The preserved clock period is 2.983 ns. The register-to-register path has 0.396 ns clock-to-Q delay, 2.456 ns combinational delay, and approximately 0.131 ns setup time:

$$T_{\min} \simeq 0.396 + 2.456 + 0.131 = 2.983\,\text{ns}. \tag{5}$$

The saved slack is displayed as `0.000 (VIOLATED)`. Thus this is a rounded zero-slack boundary, not a demonstrated positive-margin operating point. From the assignment formula,

$$WNS(10) \simeq 10 - 2.983 = 7.017\,\text{ns}, \tag{6}$$

$$F_{\max} \simeq \frac{1000}{2.983} = 335.23\,\text{MHz}. \tag{7}$$

The original $T = 10$ ns run log is absent; its slack above is *inferred* from the preserved path, not quoted as a new measurement.

Table 4: Part C results. Values retain the raw report's 2.983 ns boundary instead of calculating frequency from a prematurely rounded 2.98 ns.

Metric	Value
Worst slack at $T = 10$ ns	$\simeq 7.017$ ns (inferred)
$T_{\min}$ boundary	$\simeq 2.983$ ns
$F_{\max}$	$\simeq 335.23$ MHz
Slack at saved $T = 2.983$ ns	0.000 ns, marked VIOLATED
Slack at rounded $T = 2.98$ ns	$\simeq -0.003$ ns (inferred)
Critical start register	`_109_`, `a_reg[1]`
Critical end register	`_107_`, `p[7]`
Critical-path cell count	8 logic; 10 with endpoint flops
Latency, assignment $2T_{\min}$	$\simeq 5.966$ ns
Throughput, one result/cycle	$\simeq 335.23$ million/s

The supplied explanatory table instead uses 2.98 ns and 335.57 MHz. Its frequency is the reciprocal of that rounded period. Using the raw saved period gives the values in table 4. At 2.98 ns the implied slack is about -0.003 ns, so rounding down does not establish timing closure. Rounding up to 2.99 ns would give an estimated small positive margin, but no 2.99 ns run is supplied or claimed.

4.2 Critical path and its relation to the array

The first path in the STA file is a `rst_n` recovery check with 2.211 ns slack. It is not the critical arithmetic setup path. The relevant block is `Path Group: clk`, from `_109_/Q` to `_107_/D`. The mapped register connections identify these as `a_reg[1]` and `p[7]`. Table 5 lists every cell in order, including the two endpoint flip-flops.

Table 5: Ordered critical-path cells from the raw STA report. Types omit `sky130_fd_sc_hd__`. Delay is the reported incremental cell delay.

Step	Instance	Type	Delay (ns)
Launch	`_109_`	`dfrtp_1`	0.396
1	`_054_`	`a22o_1`	0.248
2	`_055_`	`and3_1`	0.181
3	`_058_`	`nor3_1`	0.276
4	`_070_`	`xnor3_1`	0.453
5	`_071_`	`and3_1`	0.209
6	`_085_`	`maj3_1`	0.361
7	`_097_`	`maj3_1`	0.379
8	`_098_`	`maj3_1`	0.349
Capture	`_107_`	`dfrtp_1`	setup 0.131

All eight combinational cells participate in the optimized addition network, with partial-product formation folded into its first cells. Tracing their functions back to the RTL crosses seven structural full-adder functions: `fa1_1`, `fa2_0`, `fa2_1`, `fa3_0`, `fa3_1`, `fa3_2`, and `fa3_3`. The first two mapped cells form pieces of the row-1 sum logic; subsequent cells propagate a row-2 carry and sum, then the row-3 carry chain to bit 7. Majority cells implement the full-adder carry function [11]. Eight mapped gate delays are therefore not literally eight full-adder delays. The assignment's estimate $2N = 8$ is a useful structural scale, while the library-based path accounts for actual gate types, loads, and clock-to-Q/setup overhead.

5 Part D: physical design and GDSII

5.1 Configuration, flow, and clock attempts

The delivered configuration uses design `mult_array`, both RTL files, clock port `clk`, absolute floorplan sizing, and `DIE_AREA=[0,0,150,150]`. The die area is therefore 22 500 µm^2. The required initial period is

$$T_{\text{initial}} = 0.5 \left\lceil \frac{1.2(2.983)}{0.5} \right\rceil = 4.0\,\text{ns}. \tag{8}$$

The final `CLOCK_PERIOD` in `pnr/config.json` is 6.5 ns. The supplied message screenshot records the six attempts in table 6. It is an informal record; the separate run directories and logs are not supplied, so exact failing slacks and failure corners cannot be reconstructed.

Table 6: Recorded physical-design attempts, transcribed from the supplied trial-history image. The final result is also supported by the final metrics.

Attempt	Target period (ns)	Recorded result
1	4.0	Failed timing
2	4.5	Failed timing
3	5.0	Failed timing
4	5.5	Failed timing
5	6.0	Failed timing
6	6.5	Passed

LibreLane runs synthesis, floorplanning and power-grid construction, placement, clock-tree synthesis, routing, parasitic extraction, and final checks before GDS output [7]. In this project, the added clock buffers, final routed wirelength, extracted-corner timing, and GDS cell references support those implementation stages. They do not establish the unrecorded version or exact console transcript of the original run.

Table 7: Part D results from the final configuration, metrics JSON, GDS, and recorded trial history.

Metric	Value
Final clock target	6.5 ns (153.85 MHz)
Number of recorded runs	6; individual logs absent
Core area (μm^2)	17759.5
Cells after standalone synthesis	70
Cells after place and route	4822 (all standard-cell instances)
Added kinds	Decap, filler, tap, clock and timing-repair buffers
Final routed wirelength	2612 µm
Worst setup slack, all corners	+0.499256 ns
Post-route F_{max}, all corners	166.65 MHz
Part C F_{max}, typical	$\simeq$ 335.23 MHz
DRC errors	Route 0; Magic 0; KLayout 0
LVS errors	0

5.2 Instance counts and physical overhead

The metric `design__instance__count` is 4822. Of these, `design__instance__count__stdcell` is 336 and the reported `fill_cell` class is 4486. Here that 336 subtotal includes tap cells; it is not the count of arithmetic logic alone. The GDS was independently parsed for top-level SKY130 cell references to obtain table 8. The 4486 fill-class instances split into 4214 decaps, 211 `fill_1` cells, and 61 `fill_2` cells.

Decaps add local supply capacitance, fillers complete otherwise empty row segments, and taps connect well/substrate regions to the supply rails. Clock buffers distribute the clock, while repair cells adjust timing and drive conditions; one hold buffer is reported within the 20-cell repair category. The 16 physical flip-flops use `dfrtp_2`, whereas the standalone synthesis netlist uses `dfrtp_1`, showing that the physical flow also changes drive strengths. Its 52 combinational cells differ from the original 54, so the final design is not simply the original netlist plus inserted cells.

Table 8: Final physical instances: metric classes checked against GDS names. The hold-buffer count is a subset of timing-repair cells, not another row to add to the total.

Category and observed GDS type	Count
Decaps: `decap_3`	4214
Fillers: `fill_1` and `fill_2`	272
Taps: `tapvpwrvgnd_1`	245
Timing-repair delay/buffer cells	20
`clkdlybuf4s25_1`	19
`dlygate4sd3_1`	1
Clock-tree buffers: `clkbuf_16`	3
Sequential cells: `dfrtp_2`	16
Other combinational cells	52
Total (counting categories once)	4822

The 70-to-4822 increase is dominated by physical support cells, not more multiplier arithmetic. Excluding fillers and decaps gives 336; excluding the 245 taps as well gives 91 logic, register, and buffer instances. The 1525.21 μm^2 standard-cell area subtotal is 8.59% of the 17 759.5 μm^2 core area under the metric's classification. The fixed large die explains why support cells dominate this small arithmetic block. Counting all GDS references indiscriminately would also include 750 via references and give 5572, which is not the standard-cell count used in the results table.

5.3 Routed timing and comparison by corner

Using the required formula with the overall worst setup slack,

$$T_{\text{min,post}} = 6.5 - 0.499256 = 6.000\,744\,\text{ns}, \quad (9)$$

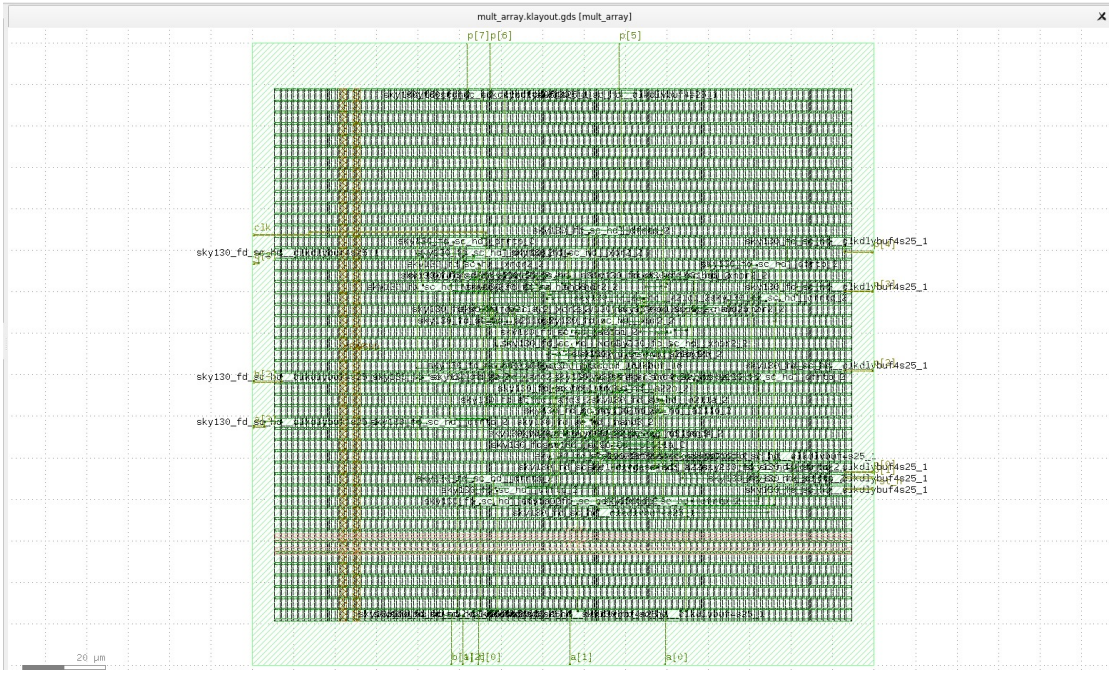
$$F_{\text{max,post}} = \frac{1000}{6.000744} = 166.65\,\text{MHz}. \quad (10)$$

This is an estimate from the 6.5 ns run, not a separately closed run at the calculated limit. The target run operates at 153.85 MHz. The worst setup entry is `max_ss_100C_1v60`, a slow, 100 °C, 1.6 V corner. Comparing 166.65 MHz with Part C's 335.23 MHz gives a 50.29% reduction, but it combines both physical implementation effects and a change of corner.

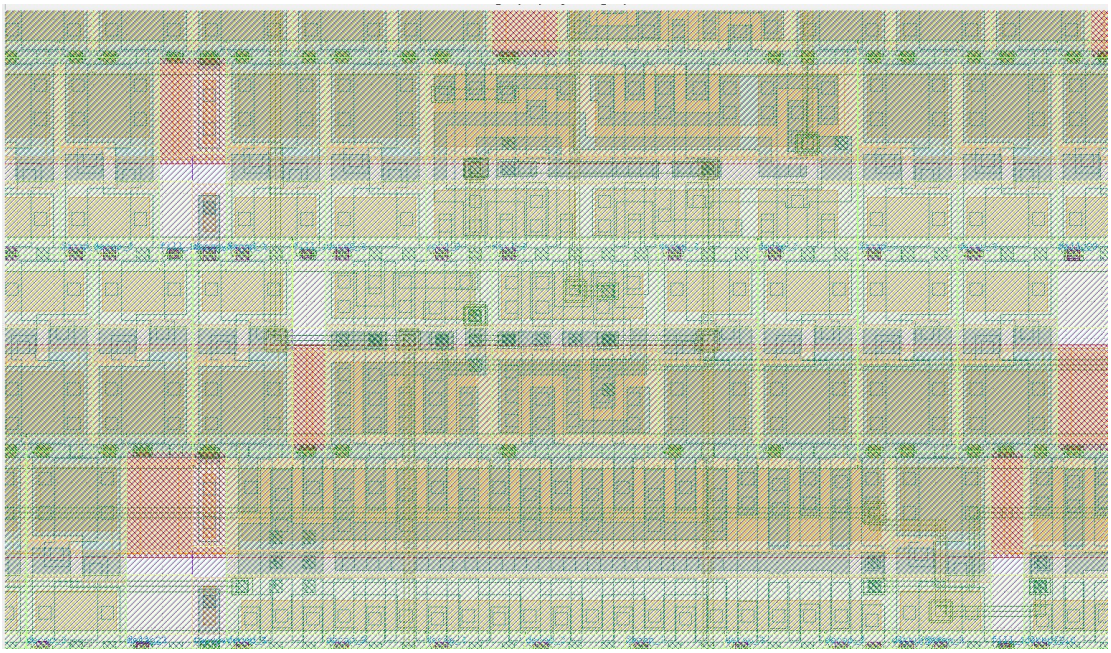
For a closer typical-corner comparison, the final `max_tt_025C_1v80` setup slack is 3.415 555 ns, giving

$$F_{\text{max,post,TT}} = \frac{1000}{6.5 - 3.415555} = 324.21\,\text{MHz}. \quad (11)$$

This is about 3.29% below the standalone typical estimate. It still includes the physical flow's remapping,



(a) Whole-die KLayout view, including the scale bar and peripheral pins.



(b) Zoomed layout showing repeated cell rows and routed metal geometry.

Figure 5: Supplied final-layout screenshots of `mult_array.klayout.gds`. Both images are kept large across the two-column page. Dense original labels and layer patterns limit visual separation of individual wires.

clock tree, and routed parasitics; it is a more comparable figure rather than a controlled experiment isolating wire delay. The metrics include nominal, minimum, and maximum extraction corners.

5.4 Layout inspection and final checks

Figure 5 contains the required whole-die and zoomed KLayout views. The whole-die image shows the core inside the die boundary, regular cell rows, and surrounding pins. The closer image shows repeated cell boundaries and overlapping routed geometry. The supplied layer hatching and labels make individual nets difficult to follow, so no exact cell identity or wire delay is inferred visually from these pictures. The quantitative counts instead come from the metrics and parsed GDS.

The router's recorded DRC iterations contain 14, 2, and finally 0 errors. The final route, Magic, and KLayout DRC metrics all equal zero. LVS reports zero device, net, property, and unmatched-object errors. Antenna violations and critical disconnected-pin counts are also zero. Overall setup slack is positive and worst hold slack is 0.118 873 ns, with no reported setup or hold violations.

The remaining electrical issue is slew: maximum-slew violation count is 11 in each reported slow-corner extraction entry. These are repeated corner entries, not evidence of 33 distinct failing nets. Positive setup slack and zero DRC/LVS errors therefore meet the assignment's stated pass conditions, but do not justify saying every electrical check is clean. The affected pins and warning details require the missing per-step reports.

6 Answers to the report questions

Q1. Why is the logic-cell count not $16 + 12$?

Synthesis produces $70 - 16 = \mathbf{54}$ combinational cells, rather than $16 + 12 = 28$. A full adder is a function implemented by several gates, while partial products and adder logic can be combined or simplified after flattening. The result contains NAND, NOR, XOR/XNOR, majority, and complex cells rather than twelve preserved full-adder macros. Constant carry inputs and Boolean sharing also change the mapping, so 54 physical logic cells is consistent with 16 AND operations and 12 structural adders.

Q2. What is the critical path?

The critical setup path runs from `_109_` ($a_{\mathrm{reg}}[1]$) to `_107_` ($p[7]$), through the eight combinational cells listed in table 5; including its endpoint flip-flops gives ten cells. The combinational delay is about 2.456 ns, and clock-to-Q plus setup brings the period boundary to about 2.983 ns. The path crosses functions associated with seven structural full adders after optimization. The estimate $2N = 8$ full-adder delays gives the expected scale, but eight mapped gates cannot be equated to eight full adders with identical delays.

Q3. Why is the flip-flop area fraction large?

The flip-flop fraction is $400.384/797.0144 \times 100 = \mathbf{50.24}\%$. Even this small multiplier needs eight input-register bits and eight output-register bits, and each mapped asynchronous-reset flip-flop occupies 25.024 μm^2. The four-bit arithmetic network is small enough that this fixed register overhead is comparable to all of its combinational logic, so registers take roughly half the synthesized area.

Q4. Why is post-route $F_{\max}$ lower, and what was added?

The worst-corner estimate falls from the typical pre-route 335.23 MHz to 166.65 MHz after implementation, with both routed parasitics/clock-tree effects and the slower 1.6 V, 100 °C corner contributing; the final maximum-extraction typical estimate is 324.21 MHz. The GDS identifies 4214 decaps for local supply support, 272 fillers for row continuity, 245 taps for well/substrate bias, three clock-tree buffers for clock distribution, and 20 timing-repair cells, including a reported hold-buffer subset of one, for timing adjustment. These additions and cell resizing explain why the final 4822-instance layout differs greatly from the 70-cell synthesis netlist.

Q5. What do DRC and LVS verify?

DRC checks whether layout geometry obeys process rules such as minimum width, spacing, and enclosure, while LVS checks whether the devices and connections extracted from the layout match the intended netlist. This project reports zero router, Magic, and KLayout DRC errors and zero LVS errors. Both checks are needed: a layout can implement the right circuit while violating fabrication rules, or satisfy geometric rules while wiring the wrong circuit. Passing them is necessary for manufacture, although it does not replace timing, electrical, and the other required signoff checks.

Q6. What changes for an 8-bit multiplier?

Using the assignment's formulas, an 8-bit array needs $N^2 = 8^2 = \mathbf{64}$ AND operations and $N(N-1) = 8(7) = \mathbf{56}$ full adders. Relative to the 4-bit array's 16 ANDs and 12 full adders, these are increases of 4× and $56/12 \simeq 4.67\times$. The approximate critical path grows from $2(4) = 8$ to $2(8) = \mathbf{16}$ full-adder delays, roughly doubling the combinational delay if the cell delay and loading were unchanged. This is a scaling estimate, not a measured 8-bit result; mapping, fanout, and routing would need a new implementation to obtain its actual frequency.

7 Limitations and submission completeness

The report supplies all Part A–D results tables, the worked example, a 12-adder block diagram, annotated waveform, RTL and gate-level PASS evidence, a newly reproduced FAIL result, ordered critical-path cells, recorded clock attempts, layout screenshots, and all six answers. The accompanying `REQUIREMENTS.md` maps

these items to the assignment.

The original project is missing its root `README.md` with the complete historical command sequence. Commands recoverable from scripts and screenshots are listed in the report package, but exact container launch, per-run commands, and original version information cannot be reconstructed. No historical Git progress is inferred from a finished report. The delivered layout file is named `pnr/mult_array.klayout.gds`; the assignment requests `pnr/mult_array.gds`. Its contents were inspected under the original name, which is retained. The completed PDF is delivered at the requested `report/report.pdf` path.

The arithmetic checks cover every unsigned operand pair under the supplied testbench's protocol. They do not establish asynchronous reset-release safety, streaming behavior under every stimulus, or post-route timed simulation. Standalone timing remains a rounded typical-corner boundary; final frequency estimates depend on the chosen corner and constraints. These limits matter more than claiming that a functional PASS alone proves every aspect of the implemented circuit.

8 Conclusion

The delivered multiplier correctly implements all 256 unsigned 4×4 products in the exhaustive checks. Its 16 registers occupy 50.24% of the 797.0144 μm^2 synthesized area. The critical arithmetic path has eight mapped logic cells and a typical-corner period boundary near 2.983 ns. The final 6.5 ns target run has positive multi-corner setup and hold slack and zero DRC/LVS errors, while slow-corner slew violations remain. The physical instance count is dominated by decap, filler, and tap cells on the fixed die; reporting that breakdown and comparing like timing corners gives a more accurate account of the implementation.

References

[1] N. Weste and D. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed., Pearson. Chapter 11 (datapath subsystems: adders and multipliers).

[2] J. Rabaey, A. Chandrakasan and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed., Pearson. Arithmetic building blocks and timing.

[3] Icarus Verilog. `https://steveicarus.github.io/iverilog`.

[4] GTKWave. `https://gtkwave.github.io/gtkwave`.

[5] Yosys. `https://yosyshq.readthedocs.io/projects/yosys`.

[6] OpenSTA. `https://github.com/parallaxsw/OpenSTA` (manual: `doc/OpenSTA.pdf`).

[7] LibreLane. `https://github.com/librelane/librelane`.

[8] OpenROAD. `https://openroad.readthedocs.io`.

[9] KLayout. `https://klayout.de/doc.html`.

[10] Magic / Netgen. `https://opencircuitdesign.com/magic`, `https://opencircuitdesign.com/netgen`.

[11] SKY130 PDK. `https://skywater-pdk.readthedocs.io`.

[12] IIC-OSIC-TOOLS. `https://github.com/iic-jku/IIC-OSIC-TOOLS`.

A Supplementary layout view

The supplied alternate view uses different visible layers on the same named GDS. It is retained as a supplementary overview (figure 6); quantitative interpretation uses the metrics and GDS counts.

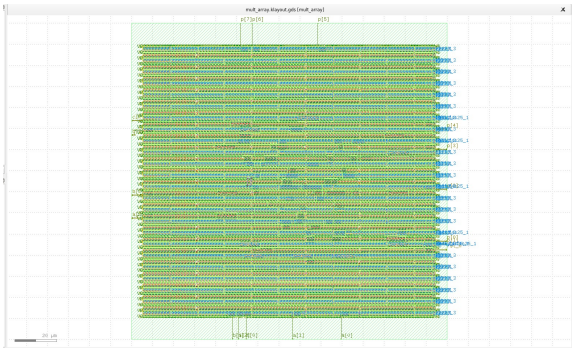


Figure 6: Alternate supplied KLayout overview with different visible layers. Individual cell identities are not inferred from this view.